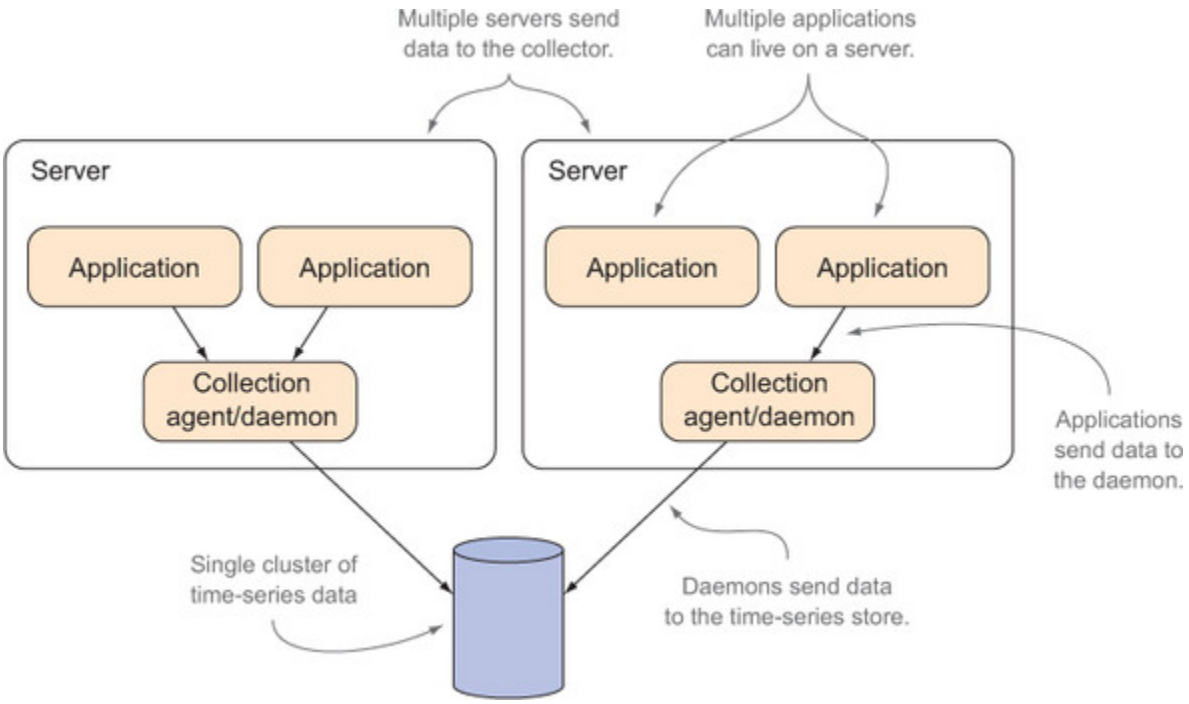


The most popular example for this is StatsD. The StatsD daemon runs on a node.js server, but there are clients you can use to send data to that daemon in many different languages. Once you have a time-series data store set up and get the daemon running, using StatsD is as simple as the code shown in the following listing.

Listing 6.1. Using StatsD

```
private static final StatsDClient statsd = new
     NonBlockingStatsDClient("ord.cwhd", "statsd-host", 8125);

statsd.incrementCounter("continuosity");
statsd.recordGaugeValue("improve-o-meter", 1000);
statsd.recordExecutionTime("bag", 25);
statsd.recordSetEvent("qux", "one");
```

Increments a counter

Sets a gauge value

Creates the static client

Records a timer

Saves an event with a timestamp

As shown in the listing, using this library is a piece of cake. Create the client object and set the metrics as you see fit in your code. The four different types of data give you lots of flexibility to record metrics as you see fit. On my teams I like to make this type of telemetry a part of quality checks that are enforced by static analysis or peer review. If you’re building code, ensure that you have the proper measurement in place to show the consumer value of the feature you’re working on.